# Generating CaloChallenge Dataset-2 Showers: Two Approaches, Their Preprocessing, Architectures and Objectives

(a) a conditional convolutional autoencoder for latent diffusion, and (b) a direct physics-informed reverse-time diffusion sampler (PINNDE)

GSoC 2026 / ML4SCI · Fast Calorimeter Simulation Challenge, Dataset 2 · compiled 19 October 2026

This document records how each approach transforms the data, what network it uses, what it optimizes, and what it measurably achieves. Both approaches target the same problem and share the same preprocessing lineage, and -- as it turned out -- the same two failure modes: an objective that did not match the reported metric, and sparse voxels that a conditional-mean regressor cannot represent. Numbers quoted are measurements from this repository's runs. Section 7 separates what has been measured from what remains untested.

## 1. The task and the data

Dataset 2 of the Fast Calorimeter Simulation Challenge contains 100,000 simulated electromagnetic showers in a cylindrical calorimeter of 45 longitudinal layers, 16 azimuthal bins and 9 radial bins: 6,480 voxels per shower. Incident energies are log-uniform from 1 GeV to 1 TeV. The task is to generate new showers conditioned on incident energy, fast enough to replace Geant4 in production.

Local file `data/Dataset2.pt` stores `showers` of shape (100000, 45, 16, 9) in MeV and `energies` of shape (100000, 1) in MeV. Properties that drive every design decision below, measured on held-out showers:

| Property | Value | Consequence |
|---|---|---|
| Voxels per shower | 6,480 | flat MLPs are 6,480-dimensional in and out |
| E_dep / E_inc | 0.78 +- 0.11 | sampling fraction; the headline calibration metric |
| Readout threshold | 15.15 keV | the data is already cut: no voxel is in (0, 15.15 keV) |
| Voxels above threshold | ~24% | ~1,500-1,600 lit voxels; 76% are exactly zero |
| Max single-voxel fraction | 0.276 | so log1p(E/E_inc) is nearly linear (see 2.2) |
| Per-voxel variance spread | ~0 to 27 | one global scale fits no single voxel well |

Table 1. Dataset-2 properties measured in this repository. The threshold being pre-applied is what makes 'voxel is non-zero' a usable supervision label for occupancy.

## 2. Shared preprocessing

### 2.1 Units

Both approaches normalize each voxel by the incident energy, so both sides of the ratio must be in the same unit. An earlier version of the pipeline divided showers in MeV by energies in GeV, inflating the target by 1000x; the log-space target range stretched about 15x (max ~6.3 instead of ~0.4), which then fed a `1 + 10*x_log` loss weighting and produced weights up to ~63 instead of ~5. Both paths now convert to GeV first. The round trip (standardize, unstandardize, expm1, multiply by energy) reproduces the raw file's E_dep/E_inc = 0.7815 to float32 precision, and checkpoints carry a `shower_units_gev` marker so consumers cannot silently re-introduce the old convention.

### 2.2 Voxel transform

The default representation, shared by both approaches, compresses dynamic range while mapping an empty voxel to exactly zero:

```
x_log = log1p(E_voxel / E_inc)        ('fraction' representation)
E_voxel = clamp(expm1(x_log), min=0) * E_inc      (exact inverse)
```

Because voxel fractions never exceed 0.276, this log is nearly the identity: the largest relative deviation between log1p(f) and f is 0.117. That matters in two opposite ways. It keeps energy sums unbiased, because a conditional mean in a near-linear space exponentiates back to approximately the right mean. But it leaves no dynamic range: typical values are ~1e-4, so a plain MSE sees almost nothing of the shower shape, and the gap between an empty voxel and a just-above-threshold hit is a median 5.5e-4 in standardized units -- twenty times below the noise floor the diffusion sampler still carries at t = 0.

The PINNDE path therefore also implements a genuinely logarithmic alternative, `log1p(E_voxel / 15.15 keV)` ('threshold'), where empty sits at 0, a threshold hit at log 2, and a 1 GeV deposit at 11.1. After standardization the empty-to-hit gap becomes 0.22 units, 400x larger. Section 5.7 shows why this helps sparsity and hurts energy, and why it is nevertheless not the default.

## 2.3 Standardization and energy conditioning

A single global scalar mean and standard deviation are applied to the whole voxel array, not per-voxel statistics: around 76% of voxels are exactly zero and many are near-always zero, so per-voxel division would amplify dead voxels into pure noise. Global scaling preserves the relative structure across voxels, at the cost of leaving per-voxel variance heterogeneous (~0 to 27), which later turns out to break one particular diffusion preconditioner (5.4).

| Quantity | Value | Where used |
|---|---|---|
| x_log mean (fraction rep.) | 1.206e-4 | global standardization, both approaches |
| x_log std (fraction rep.) | 8.10e-4 | same; the AE calls this x_scale (8.05e-4) |
| Empty voxel, standardized | -0.1489 | hit-gated read-out writes exactly this |
| x_log mean, std (threshold rep.) | 1.708, 3.127 | PINNDE alternative representation |
| Empty voxel, threshold rep. | -0.5463 | same |
| log10(E_GeV) mean, std | 1.4973, 0.8660 | energy conditioning input, both approaches |

Table 2. Preprocessing constants, all stored in the checkpoints so that sampling and evaluation cannot drift from training.

Incident energy enters both networks as a standardized logarithm, `e_cond = (log10(E_GeV) - 1.4973) / 0.8660`. How that scalar is then injected turns out to matter a great deal, and differently in the two approaches (4.1, 5.4).

# 3. How both approaches are judged

The same quantities are reported for both, always against real held-out showers measured the same way. For the autoencoder the primary number is a per-shower reconstruction error; for the generative sampler, reconstruction is undefined and distributional agreement replaces it.

| Metric | Definition | Why it is there |
|---|---|---|
| rel (AE only) | ||x_hat - x|| / ||x|| per shower, in energy-fraction space, averaged | reconstruction fidelity; the AE's reported metric |
| E_dep/E_inc | sum of voxel energies over incident energy; mean, sd and 5/50/95% | calibration. Quantiles matter: log-space outliers can dominate a mean |
| Hit fraction | fraction of voxels above 15.15 keV, and hits per shower | sparsity; a conditional-mean model fills empty voxels with small deposits |
| Profile errors | relative L1 error of the mean longitudinal and radial profiles | shower shape, separately along z and r |
| NN distance | distance to the nearest real shower in log space, vs real-to-real | much smaller means memorization; much larger means off-manifold |
| Baselines | predict zeros; predict the mean shower per energy bin; PCA at k components | a model must beat these to have learned anything (it caught a failure: 4.3) |

Table 3. Evaluation. For the sampler these are computed by `src/calods2/metrics/quick_metrics.py`; the official CaloChallenge evaluation (classifier AUC, high-level histograms) is run separately by `run_full_evaluation.py`.

# 4. Approach (a): the FactorizedCaloAE autoencoder

The autoencoder is not a generator on its own. Its purpose is to compress a shower into a small structured latent that a diffusion model can later be trained on, so the decoder must work standalone from a sampled latent. That single requirement shapes the architecture: no U-Net skip connections (they would let the decoder depend on an encoder it will not have), and the decoder is conditioned on incident energy directly, so the downstream diffusion model only has to learn shape at fixed energy.

## 4.1 Architecture

The shower is treated as 45 independent 16x9 transverse images, encoded per layer, then convolved along the longitudinal axis -- hence 'factorized'. Shapes below are as built (the transverse encoder's output shape is inferred by a dummy forward pass rather than hard-coded, which is why it is 4x3 and not the 4x2 an early comment claimed):

```
input                      [B, 45, 16, 9]   standardized log1p(E/E_inc) / x_scale
  per-layer 2D encoder     [B*45, 1, 16, 9]
    ResConvBlock2D(1->16)                    3x3 convs, GroupNorm, GELU, residual
    StridedDown2D(16->32)  -> (32, 8, 5)     stride-2 conv, learnable downsample
    ResConvBlock2D(32->32)
    StridedDown2D(32->64)  -> (64, 4, 3)
    ResConvBlock2D(64->64) -> 64*4*3 = 768 features per layer
  longitudinal 1D encoder  [B, 768, 45] -> [B, 256, 45] -> [B, 128, 45]
  bottleneck               flatten 128*45 = 5760, concat energy embedding (64)
                           Linear -> 96 -> reshape [B, 8, 12]   <- the latent
  decoder (mirror)         Linear(96 + 64 -> 5760) -> 1D 128->256->768
                           per-layer 2D: bilinear upsample + conv, 64->32->16
  output                   [B, 45, 2, 16, 9]  (v4: value channel + hit logit)
```

- **Energy embedding.** A 1 -> 64 -> 64 MLP with GELU, applied to e_cond and concatenated both at the bottleneck and at the decoder input. A raw scalar would force every downstream linear layer to learn its own scale for it.
- **Structured latent.** 8 channels over 12 longitudinal positions (96 numbers, a 67x compression) rather than a flat vector, so a 1-D diffusion model can exploit the remaining longitudinal structure.
- **Linear output, no Softplus.** In log space an empty voxel's target is exactly 0, and Softplus cannot reach 0 (its floor is log 2 ~ 0.69 at the origin).

- **Residual blocks, no skips.** Residual paths inside each block carry gradients; cross-network skips are deliberately absent so the decoder stays usable alone.

## 4.2 The v3 objective, and why training went flat after ~8 epochs

Version 3 minimized a weighted sum of five terms in log space: MSE plus a brightness-weighted L1 reconstruction, a hit/no-hit binary cross-entropy, a relative total-energy error, a physical-space MSE, and a differentiable per-shower relative error. Its hit label was x_log > 1e-4, which marks about 11.7% of voxels rather than the ~24% that are physically lit. Validation loss stopped improving around epoch 8 and sat at 0.293 through epoch 20 and beyond, with a reported reconstruction error of about 69%. Diagnosis on the saved checkpoint found four independent defects, each measured rather than inferred:

| Finding | Measurement | Interpretation |
|---|---|---|
| The model had learned almost nothing specific | AE rel 0.688 vs 0.638 for the mean shower per energy bin; PCA-8 0.605, PCA-96 0.539, PCA-256 0.461 | worse than a predictor that ignores the individual shower, and far from the best linear code of the same size: the objective, not capacity |
| Hit term fought reconstruction over one shared output | BCE stuck at 0.496, worse than a constant predictor (0.361), yet 63% of total loss; empty voxels pinned at -0.058; lit voxels at 0.52x truth | reconstruction needs ~4e-4, confident BCE needs ~+-0.3 from the same number; the two cannot both be satisfied |
| The largest gradient carried no shape information | energy-sum term gradient 0.309 vs 0.092 for relative error, cosine 0.015 between them; physical-MSE term 5.9e-7 | it moves all voxels of a shower together; the inert term wasted a slot |
| Validation scored a different loss than training | hit logit scale 1.0 in validation vs the learned 11.635 in training | and validation loss selects the saved checkpoint |

Table 4. v3 post-mortem, measured on the epoch-20 checkpoint over 2,048 validation showers. Mixed precision was excluded as a cause: rel was 0.6876 in fp32 and 0.6871 in fp16.

## 4.3 The v4 objective

Version 4 changes the output head, the input scale and the loss; the trunk is untouched.

- **Two output channels.** A value channel and a hit logit, so the classification and regression no longer share one number. At read-out, voxels the hit head calls empty are set to exactly zero energy.
- **Scaled input and output.** The network sees x_log / x_scale with x_scale = 8.05e-4 (the global standard deviation), so its inputs and value head are order 1 rather than 1e-4.
- **Loss aligned with the metric.** The main term is the squared per-shower relative error in energy-fraction space -- literally the square of the reported metric, and scale-free, so low-energy showers are not drowned out.
- **Occupancy supervision from the data's own threshold.** The hit label is 'voxel is non-zero', which is well defined because Dataset 2 is already cut at 15.15 keV.
- **A soft-gated energy term.** Total-energy error is computed on value x P(hit), matching what the gated read-out actually emits.
- **Validation uses the identical objective**, removing the train/validation mismatch.

```
rel2 = mean_b [ || expm1(v*x_scale) - expm1(x_log) ||^2 / || expm1(x_log) ||^2 ]
bce  = BCEWithLogits( h , 1[x_log > 0] )
e2   = mean_b [ ( sum_i expm1(v*x_scale)_i * sigmoid(h)_i - E_true ) / E_true ]^2

loss = rel2 + 0.1 * bce + 1.0 * e2
```

Each weight was set by a measurement, not a guess. Dropping the energy term entirely gave a slightly better reconstruction (rel 0.563 vs 0.584 at epoch 2) but a total-energy error of 0.27; applying the energy term to the

ungated values made the energy error worse than having no term at all (0.37 at epoch 1 against 0.29), because the gate removes energy the term never saw; soft-gating it fixed that.

### 4.4 Measured effect

| Epoch | v3 rel | v3 energy error | v4 rel (no energy term) | v4 rel (soft-gated) | v4 energy error (soft-gated) |
|---|---|---|---|---|---|
| 1 | 1.018 | 0.498 | 0.596 | 0.602 | 0.044 |
| 2 | 1.328 | 0.123 | 0.563 | 0.584 | 0.032 |
| 8 | 0.835 | 0.113 | not run | not run | - |
| 20 | 0.690 (best) | 0.112 | not run | not run | - |

Table 5. v4 versus v3 under identical trunk, optimizer, schedule, precision, batch size and data split; v3's logged epochs are the control. The two v4 columns are separate runs, differing only in the energy term, which costs a little reconstruction accuracy and buys an order of magnitude in energy calibration. After two epochs v4 is already better than v3 ever became, and approaching the PCA-96 reference of 0.539. Baselines on the same metric: mean shower per energy bin 0.638, all zeros 1.000.

### 4.5 Training setup and operational notes

- AdamW, learning rate 3e-4, weight decay 1e-5; cosine schedule with a 5-epoch linear warmup whose length is the total epoch count -- so changing the epoch budget changes the schedule, not merely the stopping point.
- Mixed precision (fp16 autocast with gradient scaling), gradient-norm clipping at 1.0, batch size via the `CALO_BATCH_SIZE` environment variable (set at import time; 128 was used, the default is 4096).
- A best-validation checkpoint plus a `<name>_last.pt` written every epoch. Before this, resuming fell back to the last *best* epoch and silently discarded everything since (eight epochs were lost that way).
- `train_ae_workstation.sh` relaunches on crash and stops on a clean exit. Its predecessor compared the best checkpoint's epoch against the target, so a run whose best epoch preceded its last would relaunch forever.

## 5. Approach (b): the direct PINNDE sampler

This path generates showers directly in the 6,480-dimensional voxel space: no autoencoder, no latent. It ports the formalism of the author's GSoC 2025 PINNDE project -- a physics-informed network solving a reverse-time diffusion equation -- from 1-3 dimensional Gaussian-mixture targets to real calorimeter showers with energy conditioning.

### 5.1 The formalism

A stochastic interpolant connects data to noise, with a small floor so that t = 1 is essentially a standard Gaussian:

```
x(t)     = alpha(t) * x0 + sigma(t) * eps,    eps ~ N(0, I)
alpha(t) = 1 - t,     sigma(t) = sigma0 + (1 - sigma0) * t,     sigma0 = 1e-2

reverse-time ODE:   dx/dt = G(t, x, E) = [ (1 - sigma0) * x - q(t, x, E) ] / sigma(t)
where               q(t, x, E) = E[ x0 | x(t) = x, E ]     (the posterior mean)
```

Integrating that ODE from t = 1 down to t = 0 turns noise into a shower. Everything reduces to knowing q. The reference implementation estimates it by Monte-Carlo importance weighting over a fixed ensemble of real showers, with a Gaussian kernel in voxel space and an extra energy-proximity factor so the estimate is conditional on incident energy (`flow_de.qVectorField`).

### 5.2 The original PINN residual port, and why it does not scale

The reference recipe makes the network represent the whole trajectory, x(t, z, E), with the boundary condition hard-wired by the ansatz, and trains it on a supervised term against precomputed exact trajectories plus a collocation residual of the ODE:

```
ansatz     x(t, z, E) = z + (1 - t) * net(t, z, E)        (exact at t = 1 by construction)
residual   f = t * dx/dt - t * G(t, x, E)                 (autograd through the ansatz)
loss       MSE_u (anchor trajectories) + 10 * MSE_f (collocation points)
```

At 6,480 dimensions this fails, and the failure is quantitative rather than a matter of tuning:

- **The flow map does not generalize.** Going from 512 to 4,096 anchor trajectories (8x the data) cut held-out error only 1.84x, i.e. error ~ N^-0.29. Reaching a usable error would need on the order of 1e6 times more trajectories.
- **The residual cannot train from a cold start.** It is evaluated at the model's own prediction, which early in training is off the data manifold, where the kernel estimate degenerates and G ~ x/sigma(t) becomes radially repulsive. MSE_f *rose* during training (6,862 to 22,002): it could not minimize even its own objective. A warm start from a data-fitted model did not rescue it.
- **The two terms are not mutually inconsistent.** The residual evaluated on exact trajectories is ~1e-12, so both terms share a minimizer; the problem is the optimization landscape away from it. No reweighting across four orders of magnitude helped.
- **The exact Monte-Carlo solver is itself a memorizer.** Run as an ODE with a finite reference ensemble it lands 0.4 from a training shower where genuinely new showers sit at 47 -- so it is also a poor teacher to distill.

Consequence: samples were worse than noise, with E_dep/E_inc of 2.9-3.5 against a true 0.78. This path is retained in `networks_calo.py` for reference and for the low-dimensional regime where it was validated.

## 5.3 What replaced it: learn the field, keep the physics exact

The fix keeps the interpolant, the reverse-time ODE and the posterior-mean definition, and changes only what the network represents: the *local* field q(t, x, E) instead of the global flow map. G is then recovered by exact algebra from q, so the ODE is satisfied by construction and there is no residual left to penalize. The physics lives in the interpolant and in the exact expression for G, not in a soft penalty -- worth stating plainly, since the term 'PINN' usually implies the latter.

Training targets the clean shower x0 that generated x(t): the minimizer of that mean squared error is exactly the posterior mean, so no reference ensemble is needed at all and every training point lies on the interpolant manifold by construction. Against the Monte-Carlo target, the clean target won on every diversity metric (spread across showers 0.276 vs 0.149-0.176 for a real 0.408; pairwise distance 58 vs 24-27 for a real 86; distance to the nearest training shower 32 vs 7-9 for a real 47) and trained 2-4x faster. Enlarging the reference ensemble did not rescue the Monte-Carlo target: 128 to 512 references moved the spread only 0.149 to 0.176 at 2.3x the cost. An earlier version of the project README claimed the ensemble size was the main quality knob; that claim was wrong and has been retracted.

## 5.4 Architecture

```
input      t (scalar), x (6480), e_cond (scalar)
trunk      Linear(6482 -> 1024) - SiLU - Linear(1024 -> 1024) - SiLU
           - Linear(1024 -> 1024) - SiLU - Linear(1024 -> 12960)
output     [ q-value block (6480) | hit-logit block (6480) ]

G_hat(t, x, E) = [ (1 - sigma0) * x - q_hat(t, x, E) ] / sigma(t)      exact algebra
```

- **The hit head.** A second block of 6,480 outputs predicting 'is this voxel lit in x0'. It does *not* enter the ODE -- G is built from q alone -- it only gates the final read-out at t = 0, writing the exact empty value where it says empty. This is the same fix as the autoencoder's, for the same reason: a conditional mean places an uncertain voxel at an intermediate value, which in any continuous representation is a small spurious deposit.
- **FiLM conditioning (`--cond film`).** Sinusoidal features of (t, log E) at 16 frequencies through a 128-unit MLP, producing a per-layer scale and shift on every hidden activation, with the modulation zero-initialized so training starts at scale 1 and shift 0. Same depth and width as the concatenating trunk (22.8M against 22.0M parameters). Motivation and effect in 5.7.
- **EDM preconditioning (`--precond`, off).** The standard skip-connection parameterization assumes a single data standard deviation. Here per-voxel variance spans ~0 to 27, so in near-dead voxels the skip injects noise the trunk must cancel by reproducing it -- the identity-through-a-bottleneck problem from the other side. Measured: E_dep/E_inc 2.68 +- 0.09 against a real 0.78, which is roughly unit noise left in every voxel. Retained only for comparison.

## 5.5 Objective and optimization

```
x_t = (1-t) * x0 + sigma(t) * eps        t ~ U(0,1), eps ~ N(0,I), fresh every batch

loss = || q_hat(t, x_t, E) - x0 ||^2  +  0.1 * BCEWithLogits( h , 1[x0 > empty] )
```

Adam at a peak learning rate of 2e-4, 500-step linear warmup then cosine decay to zero at the step budget, gradient clipping at 1.0, and an exponential moving average of the weights (decay 0.999) which is what gets sampled and what the checkpoint stores as its model state. The schedule is not cosmetic: at constant learning rate the samples diverged to infinity by 16k steps. Because the budget sets the schedule length, raising it is a different training regime rather than simply a longer one.

## 5.6 Sampling

- **Heun's method** (second order) on a power time grid t_k = (1 - k/T)^2, concentrating steps near t = 0 where the ODE is stiffest (|lambda| ~ 1/sigma0 = 100).
- **Denoised read-out.** The sample returned is q_hat(0, x), not the raw end point, which still carries sigma0 = 0.01 of noise in every voxel. With the exact Monte-Carlo field this step alone moved the hit fraction from 0.58 to 0.25 against a real 0.25.
- **Gate choice** (`--gate`), applied at the read-out and therefore changeable without retraining: *topk* (default) keeps the k most probable voxels with k = round(sum of the head's probabilities); *threshold* keeps probability above 0.5; *sample* draws each voxel independently.
- **Divergence handling.** About 0.23% of noise draws run away, leaving a voxel above the incident energy. These are detected per shower and re-sampled with fresh noise; whatever survives is clamped to the physically possible maximum, and the counts are always printed (5.8).
- **Export.** The 15.15 keV threshold is applied and the result written in CaloChallenge HDF5 layout. With the gate active the threshold now removes almost nothing, because empty voxels are already exactly zero.

## 5.7 Measured results

All rows below are 2,048 generated against 2,048 real held-out showers, same evaluation code. The first table isolates the two changes that fixed sparsity and calibration.

| Configuration (16k steps) | hit frac | E median | E 5-95% | long. | radial |
|---|---|---|---|---|---|
| fraction rep., no gate* | 0.777 | - | - | - | - |
| threshold rep., no gate | 0.426 | 0.321 | 0.02 - 2.66 | 0.312 | 0.346 |
| threshold rep. + gate | 0.217 | 0.321 | 0.02 - 2.58 | 0.319 | 0.316 |
| fraction rep. + gate | 0.215 | 0.725 | 0.35 - 1.15 | 0.141 | 0.320 |
| real | 0.234 | 0.782 | 0.59 - 0.97 | - | - |

Table 6. *that row predates the hit head and was measured in an earlier configuration (8k steps, EMA weights), so only its hit fraction is comparable. The hit gate cut spurious hits from 0.78 to 0.215 (the logarithmic representation accounts for roughly half of that on its own). But the logarithmic representation biases energy badly low: a denoiser predicts a conditional mean in whatever space it works in, and exponentiating a mean-of-logs undershoots (Jensen), giving a median 0.321 against 0.782 with a 0.02-2.66 spread. Once the gate handles sparsity, the logarithm is no longer needed and the near-linear representation is the better default.

| --gate | hits/shower | E mean | long. | radial | NN / real-to-real |
|---|---|---|---|---|---|
| threshold (p > 0.5) | 1359 | 0.718 | 0.159 | 0.321 | 1.11 |
| sample (Bernoulli) | 1502 | 0.711 | 0.143 | 0.283 | 1.37 |
| topk (default) | 1543 | 0.780 | 0.138 | 0.300 | 1.23 |
| real | ~1590 | 0.784 | - | - | 1.00 |

Table 7. Gate comparison on one 16k checkpoint. Thresholding drops every voxel the model is unsure about, losing ~15% of hits and ~8% of the energy. Independent Bernoulli draws fix the marginal count but real occupancy is spatially correlated, so they add salt-and-pepper noise -- visible as the nearest-neighbour distance rising to 1.37x real-to-real. topk matches the expected count while staying spatially coherent. Nothing wins on every metric.

| Metric | concat 16k | concat 100k | FiLM 16k | FiLM 100k | real |
|---|---|---|---|---|---|
| E_dep/E_inc mean | 0.780 | 0.793 | 0.729 | 0.709 | 0.790 |
| E_dep/E_inc sd | 0.232 | 0.256 | 0.130 | 0.132 | 0.110 |
| 1-10 GeV | 0.903<br>+- 0.349 | 0.920<br>+- 0.396 | 0.736<br>+- 0.193 | 0.681<br>+- 0.204 | 0.815<br>+- 0.174 |
| 10-100 GeV | 0.693<br>+- 0.129 | 0.710<br>+- 0.125 | 0.684<br>+- 0.087 | 0.682<br>+- 0.083 | 0.777<br>+- 0.070 |
| 100+ GeV | 0.747<br>+- 0.023 | 0.754<br>+- 0.038 | 0.769<br>+- 0.021 | 0.765<br>+- 0.025 | 0.780<br>+- 0.021 |
| hits/shower | 1543 | 1513 | 1541 | 1550 | ~1600 |
| longitudinal error | 0.131 | 0.150 | 0.105 | 0.114 | - |
| radial error | 0.294 | 0.198 | 0.228 | 0.123 | - |
| NN / real-to-real | 1.26 | 1.29 | 1.25 | 1.26 | 1.00 |
| diverged per 100k | - | 236 | - | 1 | - |

Table 8. Scaling and conditioning, all with the fraction representation and the topk gate. Scaling to 100k steps and width 2048 nearly halved the radial error -- it had been flat across every other change and was wrongly written off as an architectural limit. FiLM conditioning then halved the energy spread (0.256 to 0.132 against a real 0.110), halved the radial error again (0.198 to 0.123), and cut runaway noise draws from 236 per 100,000 to 1, which is what settles section 5.8. Per-bin spreads now sit at 1.17-1.19x the real ones instead of about twice. What remains is a systematic ~10% energy deficit, worst at low energy -- a calibration bias rather than a variance problem, and the one first-order defect left. concat had the overall energy scale right (+1.3%) and everything else wrong, so FiLM is a trade rather than a strict win.

### 5.8 Divergence: a property of the noise draw, not of the integrator

| Run | Diverged |
|---|---|
| 100,000 showers, 100 Heun steps | 236 (0.236%), all at E_inc = 1-10 GeV |
| those 236 re-sampled at 400 steps (fresh noise) | 3 |
| those 3 re-sampled at 1600 steps (fresh noise) | 0 |
| 20,000 showers, 400 steps from scratch | 45 (0.225%) |

Table 9. The last row is the control, and it overturns the obvious reading of the first three: the rate is the same at 400 steps as at 100, so the time grid is not the cause. What rescued the retried showers was the fresh noise draw, since a retry changes both at once. Roughly 0.23% of standard-normal draws land where the learned field runs away, which makes it a training-side property. A separate check confirmed the integrator is fine: re-sampling everything at 400 steps left every metric unchanged (energy sd 0.252 vs 0.256, radial 0.198 vs 0.198) at four times the cost, so 100 steps remains the default.

That conclusion has since been superseded by a better one. Repeating the 100k-step training with FiLM conditioning and sampling 100,000 showers the same way left **1** divergent shower instead of 236 (Table 8). The runaway draws were a symptom of the weak conditioning that also produced the doubled energy spread -- not a property of the integrator, and not something intrinsic to the learned field.

Operationally, one such shower is enough to make a mean E_dep/E_inc print 'inf' once exponentiated, and an early version of the exporter asserted on non-finite values -- discarding a finished 100k-step run at its last line. Export now clamps to the physically possible maximum (a single voxel cannot exceed the incident energy, i.e. x_log <= log 2) and reports what it clamped. Sampling is also decoupled from training, so any checkpoint can be re-sampled or re-gated in minutes.

## 6. Combining both: latent PINNDE

The two approaches compose. `models/1_latent_diffusion/train_latent.py` runs the identical field network and Heun sampler of section 5 on the autoencoder's 96-dimensional latent instead of the 6,480 voxels, then decodes:

```
showers --encode--> latents (96-d) --PINNDE--> latents --decode--> showers
```

Two of the voxel path's hard problems disappear by construction. The hit head is switched off, because latents are dense and the decoder reproduces sparsity itself. And there is no calorimeter geometry to learn, because the

decoder already knows it. The formalism was validated at 1-3 dimensions and measurably fails at 6,480; 96 is far closer to the regime where it works, and it shows -- the training loss reaches 1e-5, four orders of magnitude below anything the voxel-space sampler attains.

In exchange there is a ceiling that no amount of sampler quality can lift: whatever the autoencoder cannot reconstruct is simply unavailable. The `--ceiling` mode measures it directly, by encoding and decoding REAL showers with no diffusion model in the loop. Every latent-sampler result should be read against that file first, and only then against real showers.

| Metric | latent sampler | ceiling (AE reconstruction) | direct sampler (film 100k) | real |
|---|---|---|---|---|
| E_dep/E_inc | **0.782 +- 0.118** | 0.774 +- 0.120 | 0.701 +- 0.126 | 0.782 +- 0.120 |
| 5 / 50 / 95% | 0.61 / 0.78 / 1.00 | 0.59 / 0.78 / 0.97 | - | 0.58 / 0.78 / 0.98 |
| hits/shower | 1642 | 1633 | 1616 | 1558 |
| longitudinal error | **0.050** | 0.055 | 0.114 | - |
| radial error | **0.108** | 0.114 | 0.118 | - |
| NN / real-to-real | 1.30 | 1.27 | 1.29 | 1.00 |

Table 10. Latent PINNDE on the v4 autoencoder, 20k steps -- the best configuration measured in this repository. The energy distribution matches real showers in BOTH mean and spread, per energy bin as well as overall, which is the defect that resisted every fix in voxel space. Every generated number tracks the ceiling to within noise, so the latent diffusion is doing its job and what remains belongs to the autoencoder -- note that the ceiling's own NN distance is 1.27x real-to-real, so the off-manifold gap neither approach has closed is a reconstruction limit here, not a generator one.

An earlier run of this same comparison, on the broken v3 autoencoder, reached the same 'matches its ceiling exactly' conclusion -- but that test was far easier than it looked. v3's latents were nearly deterministic given the incident energy (per-dimension std 0.036-1.6 with several dead dimensions, training loss 1e-5), so reproducing their distribution was trivial. v4 uses all 96 dimensions (std 3.1-4.6) and its loss plateaus at 0.44, which is about the average posterior variance over t ~ U(0,1) and is what convergence looks like for this objective. The conclusion now rests on a real task.

The practical consequence is that the two tracks now have cleanly separated bottlenecks. The latent path now leads on every metric except the off-manifold distance, where the two tie and the limit is the autoencoder's reconstruction rather than either sampler. Its quality is bounded entirely by the autoencoder, so the lever is a larger or better one (PCA-256 reaches 0.461 and PCA-512 0.387 against the current 96-number model's 0.367, so there is headroom). The direct voxel path has no reconstruction ceiling at all and is instead limited by the ~10% energy deficit of section 5.7, which survived every sampling-side change tried. Which one wins on the metric that counts was an open question when this section was written; section 9 answers it, and the answer is the voxel path by a wide margin -- the latent path's ceiling turned out to be the binding constraint, exactly as this paragraph feared but more severely.

One operational note worth repeating, because an aggregate metric hid it: `train_latent.py` prints the generated latents' per-dimension standard deviation against 1.0 on every run. At 600 steps that read 0.019 -- the latents had collapsed to their mean -- while E_dep/E_inc still looked healthy at 0.759, because the mean latent decodes to an average shower with roughly the right energy.

## 7. The official evaluation, and what it changed

Every number in sections 4-6 comes from this repository's own metrics. The official CaloChallenge evaluation tells a harsher and more important story, and it supersedes the optimistic reading of those sections.

| submission | cls low | cls high | cls CNN |
|---|---|---|---|
| GEANT4 (real vs real) | 0.500 | 0.499 | 0.500 |
| CaloDREAM (best) | 0.531 | 0.521 | 0.681 |
| CaloDiffusion | 0.577 | 0.591 | 0.680 |
| CaloScore | 0.595 | 0.666 | 0.795 |
| median of 17 submissions | 0.755 | 0.888 | 0.980 |

| submission | cls low | cls high | cls CNN |
|---|---|---|---|
| CaloVAE+INN | 0.907 | 1.000 | 0.993 |
| Calo-VQ (worst listed) | 0.986 | 0.994 | 0.999 |
| **latent PINNDE (this work)** | **0.9918** | 1.0000 | 0.9999 |
| **direct PINNDE (this work)** | 0.9991 | 0.9998 | 0.9999 |

Table 11. Binary-classifier AUC against Geant4: 0.5 is indistinguishable, 1.0 is perfectly separable. Both samplers sit at the bottom of the published table. The latent path is measurably better on low-level (0.9918 vs 0.9991, JSD 0.851 vs 0.982), consistent with its better calibration, but neither is competitive. Note the pattern among the published entries: every autoencoder/VAE/VQ-based method lands at 0.90-0.99, while the leaders are direct diffusion in voxel space with geometry-aware convolutional networks.

This does not contradict the physics metrics; it supersedes them. Correct mean energy, correct hit counts and 5-11% profile errors are all AGGREGATES over voxels. A classifier looks at individual voxels. The warning was already in the data -- samples sit 1.26-1.30x further from real showers than real showers are from each other -- and the quick metrics had no view of the per-voxel energy distribution at all.

## 7.1 The defect the aggregates could not see

A units problem was ruled out first, directly against the file the evaluation uses: incident energy distributions agree, E_dep/E_inc agrees to 1%, empty voxels are exact zeros in both, no voxel falls in the forbidden (0, 15.15 keV) band, and the smallest non-zero voxel lands on the readout threshold (0.01530 generated vs 0.0151516 reference). A float32 check also cleared the representation: a threshold voxel sits 1054 float32 ticks above empty even at E_inc = 1 TeV. What is wrong is the VOXEL ENERGY DISTRIBUTION:

| non-zero voxel MeV | direct | latent (v3 AE) | v3 ceiling | reference |
|---|---|---|---|---|
| 5th percentile | 3.88 | 1.63 | 1.45 | **0.632** |
| median | 17.7 | 48.9 | 42.6 | 16.45 |
| 95th percentile | 292 | 761 | 764 | 299 |
| fraction of hits < 1 MeV | **0.006** | - | - | **0.072** |
| hits/shower | 1543 | 654 | 647 | 1599 |

Table 12. The generated showers put the right total energy in roughly the right places, but the individual voxel energies are wrong: dim voxels are 12x too rare. Real showers carry 7.2% of their hits below 1 MeV, mostly at the shower edges; the direct sampler produces 0.6%. That is trivially detectable by a classifier and invisible to every aggregate metric. quick_metrics.py now reports this row.

The hit gate turns out to be part of the mechanism but not the cure. A sweep over the four gate rules on one checkpoint (sample-time only, no retraining) shows the trade clearly: 'topk' ranks voxels by probability and probability tracks brightness, so it drops precisely the dim population (dim fraction 0.007); 'sample' and the newly added 'gumbel' rule restore it (0.045 and 0.056 against a real 0.072) but scatter those voxels independently, which costs energy (0.53-0.65 against 0.71) and manifold distance (1.35-1.48 against 1.24). No rule wins on every metric, because the gate only chooses WHICH voxels are lit -- the values the model assigns to marginal voxels are too bright regardless, and that is a training-side problem.

## 7.2 Tooling added in response

- `compare_to_reference.py` -- compares a generated file with the exact reference the evaluation uses, flagging scale mismatches, sparsity differences and threshold/rounding artifacts. Any of those would make an AUC meaningless as a measure of shower quality; none were present.
- `tests/test_selftest.py` -- 21 CPU regression tests, seconds to run, one per bug that cost a real run: units round trips, the empty-voxel mapping, G's algebra, gate semantics under non-finite logits (which once killed a 100k sampling pass with a CUDA assert), export clamping, and checkpoint buffers. It caught a Gumbel-gate precedence bug immediately.
- `src/calods2/metrics/sweep.py` -- grid sweeps that separate train-time parameters (width, steps, batch, conditioning) from sample-time ones (gate, read-out, integration steps), so read-out comparisons reuse a checkpoint and cost seconds instead of a retrain. Emits one markdown/CSV table sorted by closeness to real.

- `ae_baselines.py` -- the autoencoder against zeros, the per-energy mean shower and PCA-k, with a verdict on whether a plateau is worth leaving.
- `--latent-channels/--latent-length` on the autoencoder trainer, so latent size is sweepable; it was hard-coded at 8x12.

## 7.3 Next steps, in the order the measurements support

**Direct PINNDE.** The per-voxel energy distribution is the binding defect, and a width sweep says it is the architecture rather than the capacity. Across a 4.6x parameter range, at a fixed 16k steps, FiLM conditioning and the topk gate:

| hidden | params | ratio | sd | hits | dim_frac | vox5 | long | radial | nn |
|---|---|---|---|---|---|---|---|---|---|
| 512 | 10.9M | 0.681 | 0.101 | 1566 | 0.0050 | 3.55 | 0.133 | 0.214 | 1.218 |
| 1024 | 22.8M | 0.699 | 0.121 | 1643 | 0.0065 | 3.36 | 0.108 | 0.190 | 1.249 |
| 2048 | 49.8M | 0.702 | 0.132 | 1506 | 0.0064 | 3.42 | 0.112 | 0.184 | 1.237 |
| **real** | - | **0.783** | 0.115 | 1561 | **0.072** | **0.63** | - | - | 1.00 |

Table 13. Width sweep (src/calods2/metrics/sweep.py). The dim-voxel fraction is FLAT at 0.005-0.0065 against a real 0.072, and the 5th-percentile voxel energy is flat at 3.4-3.6 MeV against a real 0.63, across 4.6x more parameters. Profile errors improve from 512 to 1024 and then stop. Width is saturated at ~1024: the flat MLP's limit is its lack of spatial structure, not its size. (Steps are a separate axis -- the 100k-step run reaches radial 0.123 where 16k plateaus at 0.184.)

| Metric | MLP 16k (any width) | MLP film 100k | conv 8k | real |
|---|---|---|---|---|
| dim_frac | 0.0050-0.0065 | ~0.006 | **0.0326** | 0.072 |
| vox5 (MeV) | 3.36-3.55 | - | **1.47** | 0.63 |
| longitudinal error | 0.108-0.133 | 0.114 | **0.036** | - |
| radial error | 0.184-0.214 | 0.123 | **0.037** | - |
| E_dep/E_inc | 0.68-0.70 | 0.709 | **0.760** | 0.783 |
| spread | 0.101-0.132 | 0.126 | **0.107** | 0.115 |
| nn / real-to-real | 1.22-1.25 | 1.29 | 1.324 | 1.00 |
| parameters | 10.9-49.8M | 49.8M | **2.47M** | - |

Table 14. The conv trunk, trained for only 8k steps with 2.47M parameters, against MLPs at 16k-100k steps and up to 49.8M. Profile errors fall 3-5x, the energy deficit goes from 10% to 3% with a spread that matches real for the first time, and the dim-voxel fraction -- flat at 0.005-0.0065 across the ENTIRE width sweep -- moves to 0.0326. That last row is what makes this attributable to spatial structure rather than capacity. The nearest-neighbour ratio is the one regression. It costs ~20x more time per step on a small GPU and ~16x larger activations, so sampling needs a smaller chunk.

That makes the next step a measured conclusion rather than an analogy with the published leaders: replace the trunk with a geometry-aware 3D convolutional U-Net over (z, phi, r). A flat MLP has no notion of neighbourhood, which is exactly what the dim edge voxels and the per-voxel correlations require. Everything measured to work here -- the hit head, FiLM conditioning, the fraction representation, cosine LR with EMA, the Heun sampler and the divergence handling -- is orthogonal to the trunk and carries over unchanged.

**Autoencoder.** It is converged at rel 0.367 and beats PCA-512 with 96 numbers, but the latent sampler matches its ceiling exactly, so the autoencoder is the binding constraint on that whole path -- including its own classifier AUC, which the ceiling file should be measured on directly. PCA-256 reaches 0.461 and PCA-512 0.387, so a 256-number latent has measured headroom; that is now a one-flag sweep. The decoder's own voxel-energy distribution (Table 12, 'v3 ceiling' column) should be checked on v4 before investing further, since any defect there is inherited by every sample the latent path produces.

## 7.4 Three changes taken from the leading submissions

The two best Dataset-2 submissions were read for methodology: CaloDREAM (arXiv:2405.09629, SciPost Phys. 18, 088), which leads at AUC 0.531, and CaloDiffusion (arXiv:2308.03876) at 0.577. Three of their choices were missing here and have now been implemented. Two things this project already had independently are worth noting: FiLM-style affine conditioning, which CaloDREAM uses in its vision transformer, and circular padding in phi, which is

CaloDiffusion's "cylindrical convolution".

#### *Per-layer normalization and a logit transform*

Both papers normalize voxels and then apply a logit with alpha = 1e-6. Here the voxel variables become each voxel's share of its OWN layer, so every layer sums to 1, and the layer totals move into a separate set of variables. This is the piece that addresses the measured defect: under the old representation an empty voxel and a faint one differ by a median 5.5e-4 standardized units, and the model's 5th-percentile lit voxel came out at 3.4 MeV against a real 0.63. After the logit that gap is 1.35 units -- about 2500x more resolution where the model was blind. It also removes the Jensen bias that made the earlier log-of-absolute representation undershoot the energy by 2.4x, because per-voxel means can no longer move the layer totals.

#### *Two-stage factorization: energy, then shape*

```
stage A    p(u | E_inc)        45 bounded ratios: total energy and its
                               distribution along z
stage B    p(w | E_inc, u)     6480 shape numbers, each layer summing to 1
```

Both stages use the same field network and the same reverse-time ODE; only the variables differ, and the two are recombined analytically. Energy conservation stops being something the network has to learn: the ~10% energy deficit that survived every read-out, gate and representation change cannot exist here by construction. The plumbing run makes the point sharply -- at 300 steps per stage, essentially untrained, E_dep/E_inc came out 0.783 against a real 0.78, while the shape model was still producing a lit fraction of 0.47 against a real 0.25.

#### *Position channels*

The conv trunk now receives the normalized layer index and radial index as extra input channels, as CaloDiffusion does. A convolution is translation invariant by construction but a calorimeter is not, and the under-produced voxels sit at the shower edges. phi-equivariance survives because the channels vary only in z and r, and a test asserts it.

One finding from CaloDREAM confirms this project's own conclusion independently: their latent variant also overestimates the low-energy regime, with sparsity discrepancies that they attribute explicitly to the autoencoder reconstruction rather than the generative model, and mitigate with a 1 MeV cut. That is the same result as the ceiling measurement in section 6, reached by the group with the best submission.

# 8. The two-stage model: specification and measurement

Section 7.4 described the two-stage factorization as newly implemented, on the strength of a 300-step plumbing run. It has since been trained to convergence and evaluated officially, and it is now the best model in this repository by a wide margin. This section is its full specification, so it can be rebuilt from the document alone.

## 8.1 The representation

A shower is split into two blocks that are modelled independently and recombined analytically. `src/calods2/data/layerdata.py` implements both directions.

```
ENERGY BLOCK  u (45 numbers)
    u[0] = E_tot / (f * E_inc)                              in [0, 1]
    u[i] = E_{i-1} / sum_{j >= i-1} E_j     for i = 1..44     in [0, 1]
           (fraction of REMAINING energy in layer i-1;
            the 45th layer is implied by the other 44)

SHAPE BLOCK   w (6480 numbers)
    w = E_voxel / E_layer      -> every layer sums to exactly 1
           (layer totals carry NO information here)

BOTH then pass through a logit with alpha = 1e-6:
    p <- alpha + (1 - 2*alpha) * clamp(p, 0, 1)
    y  = log(p) - log1p(-p)

STANDARDIZATION
    u: per-dimension (45 active dimensions, no dead coordinates)
    w: a single global scalar (75% of voxels are exactly zero, so
       per-dimension division would amplify dead voxels into noise)
```

The constant `f` (f_scale) is set to 1.05 x max(E_tot/E_inc) so that u[0] stays strictly inside (0, 1). Measured over the full 100k training set, f_scale = 2.9756, w_mean = -11.6914 and w_std = 3.8700. These are saved in the checkpoint and must never be re-derived from a different sample at sampling time.

| Quantity | `fraction` rep. (sections 2, 5) | `layer` rep. (this section) |
|---|---|---|
| empty voxel, standardized | 0.0 | -0.5489 |
| median lit voxel, standardized | ~5.5e-4 above empty | **1.7286** |
| empty-to-lit gap | 5.5e-4 | **2.2774** |
| total-energy round-trip error | exact | **3.4e-8** (relative) |
| voxel-distribution $chi^2$ | 0.024 | **0.0025** |

Table 15. The representation change alone, with no architectural change, improves the empty-to-lit resolution by a factor of 4000 and the voxel-distribution chi-squared by 10x. This was the single highest-value change in the project. The inverse is analytic rather than learned, so energy conservation is exact by construction -- the residual 3.4e-8 is float32 round-off, not model error.

### 8.2 The two networks

Both stages are the same `VelocityNet` of section 5 and the same reverse-time ODE; only the variables and the trunk differ.

| | stage A: energy | stage B: shape |
|---|---|---|
| variable | u, 45 numbers | w, 6480 numbers |
| trunk | film MLP | conv 3D U-Net |
| hidden | 512 | 1024 |
| hit head | no (the data is dense) | **yes** |
| conditioning | E_inc only | E_inc **and all 45 u** (cond_extra = 45) |
| batch | 256 | 64 |
| steps | 20,000 | 100,000 |
| parameters | 993,325 | 2,474,242 |

Table 16. The two stages. Stage B is conditioned on stage A's output, so the factorization is p(u | E) p(w | E, u) -- a proper factorization of the joint, not two independent models. The extra 45 conditioning numbers travel in the existing energy argument, so every call signature in velocity_net.py is unchanged.

Sampling draws u from stage A, then w from stage B conditioned on [e_cond, u], then recombines them with `layerdata.decode`. Both stages use `readout="raw"` -- the ODE end point is already a sample, whereas the denoised read-out is a conditional mean diluted by occupancy uncertainty -- and stage B uses the `topk` gate.

### 8.3 Why it works: three separable mechanisms

- **Energy conservation becomes exact.** The single-stage models carried a persistent ~10% energy deficit that survived every read-out, gate and representation change tried. Here the total is one of 45 numbers a small network models directly, and the shape model cannot affect it at all. E_tot/E_inc $chi^2$ = 0.0020 against 0.127 for the film MLP: a factor of 63.
- **The logit representation becomes usable.** Once layer totals are stage A's job, the shape model works in a space with real resolution at the faint end (Table 15), and the per-voxel means can no longer bias the total energy through Jensen's inequality.
- **Each network solves an easier problem.** Stage A is a 45-dimensional density. Stage B is a normalized field with no scale freedom. Neither has to learn "how much" and "where" simultaneously.

## 9. Results: all five models, and the published comparison

All six evaluation directories have now been run through the official `evaluate.py`. The numbers below are read directly out of its output files.

| # | Model | cls low | cls high | cls CNN | mean chi$^2$ | params |
|---|---|---|---|---|---|---|
| **1** | **Two-stage PINN** | **0.578** | **0.695** | **0.804** | **0.0017** | 3.5M |
| 2 | PINN conv U-Net | 0.683 | 0.925 | 0.894 | 0.0082 | 2.5M |
| 3 | AE pre-v4 (reconstruction) | 0.989 | - | - | 0.139 | 3.1M |
| 4 | Latent diffusion | 0.991 | 1.000 | 1.000 | 0.151 | 1.0M + 3.1M |
| 5 | PINN film MLP | 0.999 | 1.000 | 1.000 | 0.182 | 22.8M |

Table 17. All models, ranked by low-level classifier AUC. mean chi-squared is the average over all 227 histogram chi-squared values. The two metrics rank the models identically, which is reassuring since they measure different things. The AE row is a RECONSTRUCTION of real showers, not a generated sample, so it is not a like-for-like entry -- read it as the ceiling for anything built on its decoder, which is exactly what bounds row 4.

**Correction on the autoencoder row.** It is the PRE-v4 model, not v4. `best_calo_ae_cond_improved.pt` is epoch 85, has no `two_head` key and no `shower_units_gev` marker, and was trained under the old log1p(E_MeV / E_GeV) convention; its own reconstruction error is 0.395. The v4 model (two-head decoder, rel 0.367 at 255 epochs, lit fraction 0.256) exists only on the workstation and has never been officially evaluated -- no checkpoint in the working copy has two_head=True. Earlier revisions of this document quoted v4's reconstruction quality next to this model's AUC and FPD; those are different checkpoints. The 41.98% lit fraction measured in the evaluated file is itself a no-hit-head signature, which is how the misattribution was caught.

A latent-dimension sweep adds an independent reading. Five sizes were trained (8, 24, 48, 96, 128 numbers) with reconstruction error falling 0.598 to 0.394 -- but the epochs are confounded (145 to 611), and all five predate the units fix. Only the 96-dimensional run was evaluated, at two epochs: AUC 0.9888 at epoch 85 against 0.9919 at epoch 460, with FPD 11328 against 12437. Reconstruction improved; the classifier did not move. Reconstruction error and classifier separability are close to decoupled, so a better autoencoder is not by itself a route to a competitive latent generator.

**Run-to-run scatter.** Two directories turned out to hold the same generated file evaluated twice, with the classifier retrained from scratch each time. That accidental duplicate is the only error bar available: cls-low 0.6700 vs 0.6827, cls-high 0.9224 vs 0.9250, cls-resnet 0.8814 vs 0.8935. The two-stage model was also evaluated twice (cls-low 0.5800 / 0.5781). So differences below about 0.015 AUC are noise, and the ordering of rows 3-5 is not meaningful -- they are all simply separable. The two-stage model's 0.105 lead over row 2 is about eight times that scatter and is real.

## 9.1 Histogram chi-squared by observable class

| Model | E_tot/ E_inc | E layer | EC eta | EC phi | Width eta | Width phi | Voxel dist. |
|---|---|---|---|---|---|---|---|
| Two-stage | 0.0020 | **0.0039** | **0.0010** | **0.0010** | **0.0010** | **0.0009** | **0.0025** |
| PINN conv | 0.0027 | 0.0139 | 0.0015 | 0.0015 | 0.0060 | 0.0065 | 0.0243 |
| AE pre-v4 | **0.0013** | 0.1866 | 0.1363 | 0.1382 | 0.1785 | 0.1852 | 0.1443 |
| Latent diffusion | 0.0146 | 0.1551 | 0.2119 | 0.2121 | 0.2004 | 0.1999 | 0.0629 |
| PINN film MLP | 0.1270 | 0.1208 | 0.2540 | 0.2541 | 0.2342 | 0.2361 | 0.0466 |

Table 18. Mean chi-squared over all layers in each class; lower is better. The two-stage model is best or tied-best in six of seven columns, losing only E_tot/E_inc to the autoencoder -- which is reconstructing a real shower's energy rather than generating one. Two patterns to read off: the conv trunk is what buys spatial accuracy (EC eta/phi improves two orders of magnitude from the film MLP to either conv model), and the layer-logit representation is what buys the voxel distribution (0.024 to 0.0025 from a change of variables alone).

Every model's worst observable is the deepest layers -- E layer 40-44, the sparse shower tail where there is least training signal and voxel-to-voxel correlation matters most. For the two-stage model that peaks at chi$^2$ = 0.019; for the film MLP, 0.78.

## 9.2 Against the 17 published submissions

Source: the CaloChallenge 2022 summary paper (arXiv:2410.21611), Tables C15 and C16, which average 10 independent classifier trainings with isotonic calibration. The same `evaluate.py` and the same three modes are used here, so the procedure matches; our numbers are single-run.

| Submission | cls low | cls high | cls CNN |
|---|---|---|---|
| Geant4 vs itself (floor) | 0.500 | 0.499 | 0.500 |
| CaloDREAM | 0.531 | 0.521 | 0.681 |
| CaloDiffusion | 0.577 | 0.591 | 0.680 |
| **Two-stage PINN (this work)** | **0.578** | **0.695** | **0.804** |
| CaloScore | 0.595 | 0.666 | 0.795 |
| SuperCalo | 0.694 | 0.757 | 0.986 |
| **PINN conv U-Net (this work)** | **0.683** | **0.925** | **0.894** |
| conv. L2LFlows | 0.708 | 0.737 | 0.941 |
| CaloINN | 0.743 | 0.865 | 0.994 |
| iCaloFlow teacher | 0.763 | 0.837 | 0.970 |
| CaloPointFlow | 0.863 | 0.908 | 0.999 |
| CaloVAE+INN | 0.907 | 1.000 | 0.993 |
| DeepTree | 0.963 | 0.927 | 0.999 |
| CaloLatent | 0.983 | 0.995 | 1.000 |
| Calo-VQ | 0.986 | 0.994 | 0.999 |
| **Latent diffusion (this work)** | 0.991 | 1.000 | 1.000 |
| **PINN film MLP (this work)** | 0.999 | 1.000 | 1.000 |

Table 19. Dataset-2 classifier AUCs, published submissions interleaved with this work (abridged: 13 of 17 published entries shown, spanning the full range). The two-stage model's low-level AUC is statistically indistinguishable from CaloDiffusion's 0.577, which placed second of seventeen.

**The honest reading, which the headline number oversells.** On low-level observables the two-stage model matches the second-best published submission and would rank 2nd-3rd of 18. On high-level observables it falls 0.104 behind CaloDiffusion, and on the CNN classifier 0.124 behind -- both roughly eight times the run-to-run scatter, dropping it to about 4th on each. The fair summary is "competitive with the best published submissions on one of three classifiers, mid-table on the other two", not "second place in the CaloChallenge".

That pattern is diagnostic rather than noise. The low-level classifier sees raw voxels; the high-level one sees layer energies and shape summaries; the CNN sees spatial structure convolutionally. Doing well on raw voxels and worse on DERIVED quantities says the per-voxel marginals are close to right while the CORRELATIONS between voxels are not -- and the deep-layer chi-squared values localize that to the sparse shower tail. This is the highest-value modelling work remaining.

### *Latent diffusion: independently corroborated as a dead end*

CaloLatent, the one published submission that also runs diffusion in a learned latent space, placed 15th of 17 at AUC 0.983. The latent path here scored 0.991 against an autoencoder reconstruction ceiling of 0.989 -- it essentially reached its own ceiling, so the sampler is not the problem, the decoder is. Its worst observables are the azimuthal energy centroids (EC phi, $\text{chi}^2$ ~ 0.445 for layers 26-29, far worse than any other model here), which says the 96-number bottleneck does not preserve phi position faithfully enough and that latent-space errors become STRUCTURED spatial errors after decoding. Two independent lines of evidence now point the same way: this path needs a better autoencoder, not a better sampler.

### *FPD and KPD: not competitive, and largely unmeasured*

| Model | KPD x$10^3$ | FPD x$10^3$ |
|---|---|---|
| Geant4 vs itself (floor) | -0.028 | 10.78 |
| CaloDREAM | 0.023 | 24.65 |

| Model | KPD x10$^3$ | FPD x10$^3$ |
|---|---|---|
| CaloScore | 0.149 | 112.48 |
| CaloDiffusion | 0.174 | 146.93 |
| CaloVAE+INN (worst published) | 45.61 | 5443.43 |
| **AE pre-v4 (this work, reconstr.)** | **91.60** | **11328.44** |

Table 20. The only FPD/KPD measurement in this repository is the autoencoder's, and it is worse than the worst published submission by a factor of two in both metrics. That is a real result, not a normalization artifact: it is consistent with the AE's occupancy defect (41.98% of voxels lit against a real 24.95%), which FPD and KPD are specifically sensitive to because they compare distributions in a feature space rather than marginal histograms.

There is no FPD/KPD for the two-stage model, which is the most important gap remaining, since these are the metrics the summary paper leads with for distributional fidelity. Given its voxel-distribution chi-squared of 0.0025 a far better number is expected -- but that is an expectation, not a measurement, and it should not be quoted as anything else until it is run.

One caveat on provenance: the published tables were extracted from the paper's PDF, whose column layout the text extractor interleaved. They were reconstructed by grouping values into per-submission triplets and cross-checking against the paper's own prose -- which states that CaloDREAM and CaloDiffusion have the lowest AUCs, that flow-based models follow, and that VAE- and GAN-based submissions are worst; all three hold, and the Geant4 floor came out at the expected 0.500. Spot-check CaloDREAM, CaloDiffusion and Calo-VQ against Table C15 directly before quoting these elsewhere.

## 10. What the two approaches share

The two designs are independent, but the same two mistakes appeared in both, and the same two fixes worked. That is the most transferable result here.

| Lesson | In the autoencoder | In the PINNDE sampler |
|---|---|---|
| Sparse voxels need explicit occupancy modelling | One output served both regression and classification; the hit BCE was 63% of the loss and unsatisfiable. A second output channel fixed it. | A conditional mean filled empty voxels: 78% of voxels lit vs a real 23%. The same second head, gating the read-out, brought it to 0.215. |
| The objective must match the reported metric | Log-space MSE/L1 on values of ~1e-4 barely saw shape. The main term became the square of the reported relative error. | A logarithmic representation biases the energy low through Jensen; the near-linear one keeps sums unbiased. |
| Conditioning strength matters | Energy is embedded (1 -> 64 -> 64) and fed to encoder and decoder, not passed as a bare scalar. | Two scalars among 6,480 inputs could not represent the energy dependence; FiLM halved the energy spread. |
| Compare against trivial baselines | rel 0.688 was worse than the mean shower (0.638) -- which is how the dead objective was caught at all. | The exact Monte-Carlo solver scores 0.4 to the nearest training shower: it copies. NN distance is reported always. |

Table 21. Shared failure modes and fixes.

Where they differ: the autoencoder is judged by per-shower reconstruction and is only half a generator -- a latent diffusion model still has to be trained on its 96-number latent, and the decoder's own errors then bound whatever that model achieves. The PINNDE sampler is a complete generator whose output is judged distributionally, at the cost of working in 6,480 dimensions directly, where it needs about 200 network evaluations per shower (100 Heun steps, two evaluations each).

## 11. Status, and what is not yet established

| Item | Status |
|---|---|
| **Best model: two-stage PINN** | **trained, sampled and officially evaluated; cls-low 0.578, mean chi$^2$ 0.0017** |
| Two-stage vs the published submissions | low-level AUC ties CaloDiffusion (2nd of 17); about 4th on high-level and CNN (Table 19) |
| Official evaluation of all five models | DONE; six evaluation directories (Tables 20-22) |
| FPD / KPD for the two-stage model | NOT run -- the most important remaining gap, since the summary paper leads with it |
| 10-run classifier averaging | NOT done; our single runs carry ~0.015 AUC scatter against the published +-0.004 |
| Voxel-to-voxel correlations | the binding defect: cls-high and cls-CNN lag cls-low by 0.10-0.12 (section 9.2) |
| Single-stage conv U-Net | cls-low 0.683, mean chi$^2$ 0.0082; mid-table, ahead of every published flow on low-level |
| Autoencoder v4 | converged: rel 0.367, beating PCA-512 (0.387) with 96 numbers; but FPD/KPD worse than the worst published submission |
| Latent diffusion | measured at its own ceiling (0.991 vs 0.989); a decoder limit, not a sampler one. Published CaloLatent agrees: 15th of 17 |
| Energy conservation | SOLVED by the factorization: E_tot/E_inc chi$^2$ 0.127 -> 0.0020 |
| Dim-voxel deficit | SOLVED by the layer-logit representation: voxel-dist chi$^2$ 0.024 -> 0.0025 |
| Divergent noise draws | 0.236% with concat, 0.001% with FiLM: a conditioning symptom, essentially gone |

Table 22. Honest status as of 19 October 2026. Three of the defects that dominated earlier revisions of this document -- the ~10% energy deficit, the 12x dim-voxel shortfall and the energy over-dispersion -- are now closed, each by a specific measured change. What replaced them is a correlation defect visible only to the high-level and convolutional classifiers.

Two corrections worth carrying forward, both of which were stated confidently here before being tested. The radial profile error was attributed to the flat MLP's lack of calorimeter geometry and declared insensitive to training -- it then nearly halved under more training and width. And the divergent showers were attributed to integrator stiffness on the strength of a retry that fixed them at 400 steps; a from-scratch run at 400 steps showed the same divergence rate, proving the fresh noise draw had been doing the work.

## 12. Reproducing these results

A full end-to-end replication guide lives in `docs/REPLICATION.md`: data conversion, every representation, every architecture with channel counts, every loss term, all hyperparameters, sampling, export, verification and eight pitfalls. What follows is the command-level summary.

### *Data*

```
# Dataset 2 from the CaloChallenge Zenodo record. Two files:
#   dataset_2_1.hdf5  -> TRAIN  (converted to data/Dataset2.pt below)
#   dataset_2_2.hdf5  -> EVALUATION REFERENCE (never train on this)

python - <<'EOF'
import h5py, torch
with h5py.File('dataset_2_1.hdf5') as f:
    sh = torch.tensor(f['showers'][:], dtype=torch.float32)
    en = torch.tensor(f['incident_energies'][:], dtype=torch.float32)
torch.save({'showers': sh.reshape(-1, 45, 16, 9),
            'energies': en.reshape(-1, 1)}, 'data/Dataset2.pt')
EOF

# BOTH arrays stay in MeV. See section 2 and REPLICATION.md 15.1 --
# converting only one of them cost this project a long plateau.
```

### *Two-stage model (the recommended one)*

```
# 21 CPU regression tests first: seconds, no GPU, no data file
pytest

# full run: stage A 20k steps, stage B 100k, then sample 100k showers
nohup env PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
  python -u -m calods2.train_two_stage --data data/Dataset2.pt \
    --checkpoint-energy checkpoints/ts_energy.pt \
    --checkpoint-shape  checkpoints/ts_shape.pt \
    --steps-energy 20000 --steps-shape 100000 --cond conv --batch 64 \
    --export generated/two_stage_ds2_100k.hdf5 \
    --n-sample 100000 --sample-steps 50 > logs/two_stage.log 2>&1 &

# --resume continues an interrupted run on the SAME schedule;
# keep --steps-* identical or the cosine LR lands on a different curve.
# --skip-energy reuses a trained stage A; both --skip-* = sample only.

# EXACTLY 100,000 showers: evaluate.py hard-codes a 60k/20k/20k split
# for dataset 2, so a 20k file fails on len(data1) == len(data2).
python -m calods2.metrics.quick_metrics --gen generated/two_stage_ds2_100k.hdf5
python run_full_evaluation.py --skip-generate \
    --gen-file generated/two_stage_ds2_100k.hdf5 \
    --out-dir evaluation_two_stage
```

### *Autoencoder*

```
# 1-epoch check on a throwaway checkpoint
EPOCHS=1 CKPT=smoke.pt PYTHON=$(which python) ./train_ae_workstation.sh

# long run (EPOCHS also sets the cosine schedule length)
nohup env EPOCHS=200 CKPT=best_calo_ae_v4.pt PYTHON=$(which python) \
    ./train_ae_workstation.sh > logs/launcher_v4.log 2>&1 &

# per-epoch record (tqdm redraws with carriage returns, hence tr)
tr '\r' '\n' < "$(ls -t logs/train_ae_*attempt*.log | head -1)" \
    | grep -aE "^Epoch [0-9]+/[0-9]+$|^TRAIN|^VALID|^LR|Saved"
```

### *PINNDE sampler*

```
# train, sample and export in one go (no anchor-generation step)
python models/2_direct_pinn/train.py --data data/Dataset2.pt \
    --checkpoint checkpoints/velocity.pt --cond film --steps 100000 \
    --batch 64 --hidden 2048 --device cuda \
    --export generated/pinnde_recon_ds2.hdf5 --n-sample 100000 --sample-batch 2048

# re-sample or re-gate an existing checkpoint, no retraining
python models/2_direct_pinn/sample.py --checkpoint checkpoints/velocity.pt \
    --data data/Dataset2.pt --export generated/gen.hdf5 --n-sample 100000 \
    --gate topk --sample-batch 2048

# fast physics check against real showers, then the official evaluation
python -m calods2.metrics.quick_metrics --gen generated/pinnde_recon_ds2.hdf5
python run_full_evaluation.py --skip-generate \
    --gen-file generated/pinnde_recon_ds2.hdf5
```

### *Latent PINNDE*

```
# the reconstruction ceiling first -- it predicts the outcome before training
python models/1_latent_diffusion/train_latent.py --ae best_calo_ae_v4.pt --ceiling \
    --export generated/latent_ceiling.hdf5 --n-sample 20000

python models/1_latent_diffusion/train_latent.py --ae best_calo_ae_v4.pt \
    --checkpoint checkpoints/latent_velocity.pt --steps 20000 --hidden 512 \
    --batch 256 --export generated/latent_recon_ds2.hdf5 --n-sample 100000

# is the plateau a good place to stop? AE against PCA and mean-shower baselines
python ae_baselines.py --checkpoint best_calo_ae_v4.pt
```

### *Source files*

| File | Role |
|---|---|
| train_auto_encoder_improved.py | autoencoder: model, v4 loss, training loop, entrypoint |
| train_ae_workstation.sh | crash-tolerant launcher for long autoencoder runs |
| generate_caloeval_hdf5.py | autoencoder reconstructions to CaloChallenge HDF5 |
| src/calods2/data/dataset.py | both representations, standardization, exact inverses |
| src/calods2/models/flow_de.py | Monte-Carlo q-field and the exact reverse-ODE solver |
| models/0_original_flowmap/networks_calo.py | the original PINN residual path (reference; does not scale) |
| src/calods2/models/velocity_net.py | the field network, hit head, FiLM, training, Heun sampler |
| models/2_direct_pinn/train.py | train, sample and export entrypoint |
| models/2_direct_pinn/sample.py | sample or re-gate a trained checkpoint; divergence report |
| src/calods2/data/export.py | physical conversion, clamping, divergence mask, HDF5 writer |
| src/calods2/metrics/quick_metrics.py | fast comparison of generated against real showers |
| models/1_latent_diffusion/train_latent.py | latent PINNDE: encode, sample, decode, and the --ceiling bound |
| src/calods2/data/layerdata.py | two-stage representation: per-layer logit, with an exact inverse |
| models/3_two_stage/train.py | energy model + shape model, recombined analytically |
| ae_baselines.py | autoencoder vs zeros / mean shower / PCA-k, with a verdict |
| compare_to_reference.py | generated vs the evaluation's own reference file; units check |
| tests/test_selftest.py | 21 CPU regression tests for the bugs that cost real runs |
| src/calods2/metrics/sweep.py | train-time and sample-time grid sweeps -> one metrics table |

Table 23. Where each piece lives. docs/PINNDE_LAB_NOTEBOOK.md carries the same measurements in tabular form next to the code.